

Lectura 7.1: Operadores Neuronales de Fourier (FNO y PINO)

Introducción a las redes neuronales informadas por física
Especialización en Inteligencia Artificial - B32025

Outline

- Intro y conceptos preliminares
- Detalles constructivos
- Ejemplos prácticos
- PI-FNO (PINO)
- Implementaciones

- **Paper: Fourier Neural Operator For Parametric Partial Differential Equation (ICLR Conference, 2021). Zongyi Li et al.**

Zongyi Li

Computing and Mathematical Science, Caltech.

<https://zongyi-li.github.io/>

I am a Postdoctoral Associate at MIT hosted by **Kaiming He**. I obtained my Ph.D. in computing and mathematical science from Caltech (2019-2025), mentored by **Anima Anandkumar** and **Andrew Stuart**. I did three summer internships at Nvidia. Previously, I obtained a B.S. at Washington University in St. Louis (2015-2019), majoring in computer science and mathematics with a minor in Jazz. I am fortunate to have received support from the Kortschak Scholarship, PIMCO Fellowship, Amazon AI4Science Fellowship, Nvidia Fellowship, and MIT Novo Nordisk AI Fellowship.

My long-term goal is to develop machine learning methods for scientific computing and discovery. Specifically, I pioneer **neural operators** for learning solution operator in partial differential equations (PDEs) arising in fluid mechanics and earth science. Neural operators model physical simulation with chaotic behaviors and complex geometries, showing applications in weather forecasting, carbon storage, and aerodynamics simulation.



Conceptos preliminares

- Las FNOs combinan operadores integrales lineales globales (via transf. de Fourier) con funciones de activación no-lineal locales → Permite aproximar operadores fuertemente no lineales con altas frecuencias
- **El aprendizaje de operadores puede concebirse como un problema de mapeo imagen-a-imagen** (por ej., la imagen de una CI a la imagen en $t>0$)
- La convolución en el dominio espacial es equivalente a una multiplicación punto-a-punto en el dominio de Fourier (frecuencia)
- Las “capas de Fourier” (Fourier layers) pueden ser vistas como un sustituto de las “capas de convolución” en CNN
- **Se obtienen operadores independientes de la malla (mesh-independent) e invariantes en la resolución (resolution-invariant)**

Conceptos preliminares

$$u = (K_l \circ \sigma_l \circ \dots \circ \sigma_1 \circ K_0) v$$

Representación abstracta
de un modelo de DL

- u y v pueden tener diferentes discretizaciones (cómo era en el caso de DeepONets??)
- K es un mapeo lineal/convolucional formulado en base a un **operador integral lineal** definido como $K : v_t \mapsto v_{t+1}$

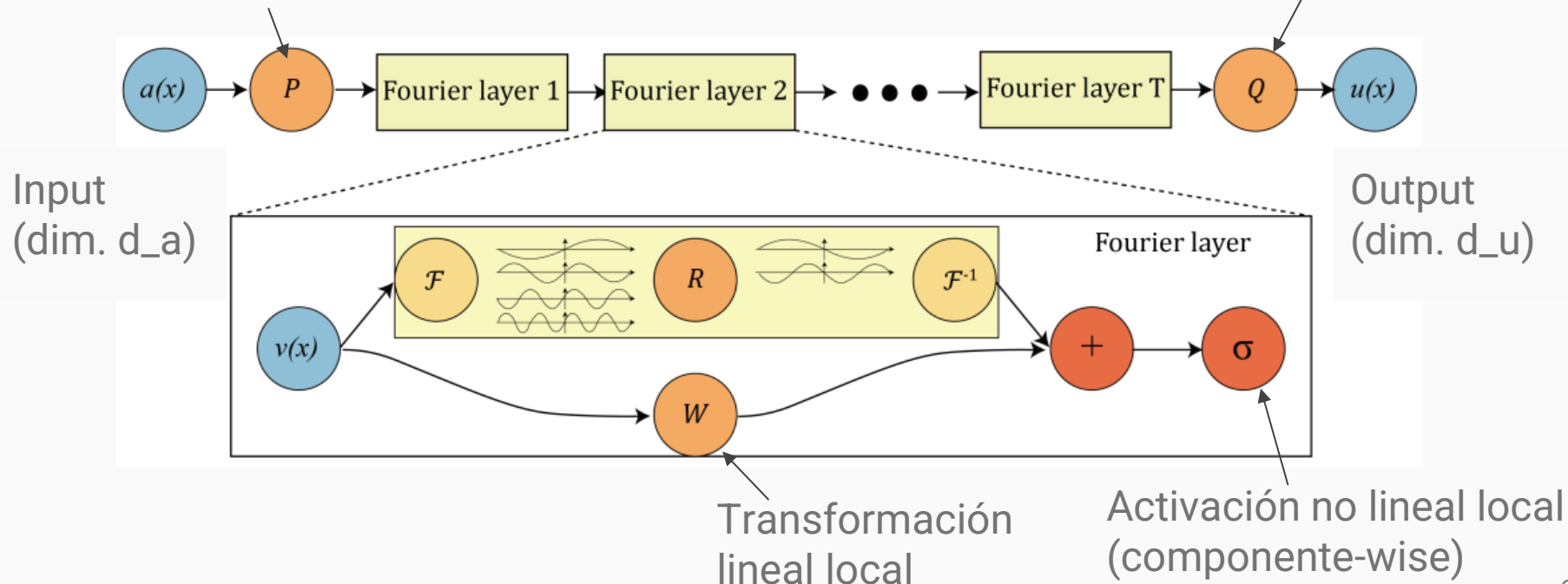
$$v_{t+1}(x) := \int \kappa(x, y) v_t(y) dy + W v_t(x)$$

- En FNO, κ es un kernel de convolución definido como $\kappa(x, y) = k(x - y)$, y se implementa a través de la transformada de Fourier (FT)

Arquitectura de las FNOs

“Lift” hacia un espacio de dimensión superior (pensar en canales)

Proyección hacia la dimensión objetivo



Arquitectura de las FNOs

Consideremos nuevamente la representación $v_t \mapsto v_{t+1}$, pero parametrizada por $\phi \in \Theta_{\mathcal{K}}$

$$v_{t+1}(x) := \sigma(W v_t(x) + \mathcal{K}(\phi) v_t(x))$$

$$(\mathcal{K}(\phi) v)(x) = \int_{\Omega} \kappa_{\phi}(x, y) v(y) dy$$

Utilizando el kernel de convolución (parametrizado por ϕ):

$$(\mathcal{K}(\phi) v)(x) = \int_{\Omega} \kappa_{\phi}(x - y) v(y) dy = (\kappa_{\phi} * v)(x)$$

Es una representación parametrizada por ϕ en el espacio de Fourier, usando FFT

Arquitectura de las FNOs

Recordemos la definición de TF y su inversa, de una función f :

$$(\mathcal{F}f)_j(k) = \int_D f_j(x) e^{-2i\pi\langle x, k \rangle} dx, \quad (\mathcal{F}^{-1}f)_j(x) = \int_D f_j(k) e^{2i\pi\langle x, k \rangle} dk$$

Teorema de convolución: *“la TF de la convolución de dos funciones es igual al producto de sus TF respectivas”*

$$\mathcal{F}\{(\mathcal{K}(\phi)v)(x)\}(k) = \mathcal{F}\{(\kappa_\phi * v)(x)\}(k) = \mathcal{F}\{\kappa_\phi\}(k) \mathcal{F}\{v\}(k).$$

v expresada en
el espacio de
Fourier
(frecuencia k)

$$(\mathcal{K}(\phi)v)(x) = \mathcal{F}^{-1}(\mathcal{F}\{\kappa_\phi\}(k) \mathcal{F}\{v\}(k))(x). \quad \text{Retorno al espacio físico}$$

Operador integral de Fourier

Se define el *operador integral de Fourier* como:

$$(\mathcal{K}(\phi)v_t)(x) = \mathcal{F}^{-1}\left(R_\phi \cdot (\mathcal{F}v_t)\right)(x) \quad \forall x \in D$$

donde $R\phi$ es la TF de una función periódica κ , parametrizada por $\phi \in \Theta_\mathcal{K}$

(entonces admite una expansión en serie de Fourier, por lo que se trabaja con los modos discretos $k \in \mathbb{Z}^d$.)

Para cada modo de frecuencia $k \in D$:

$$(\mathcal{F}v_t)(k) \in \mathbb{C}^{d_v} \text{ and } R_\phi(k) \in \mathbb{C}^{d_v \times d_v}$$

$R\phi$ se parametriza directamente como un tensor $(k_{\max} \times d_v \times d_v)$ de componentes complejas que representa una colección de modos de Fourier truncados (los demás se los hace cero)

Operador integral de Fourier discreto y FFT

- El cómputo de las frecuencias truncadas se realiza con la DFT:

$$\hat{f}_k = \sum_{n=0}^{N-1} f_n e^{-2\pi i \frac{kn}{N}}, \quad f_n = \frac{1}{N} \sum_{k=0}^{N-1} \hat{f}_k e^{2\pi i \frac{kn}{N}}$$

- La convolución entre dos secuencias “f” y “g” se realiza simbólicamente como:

$$f * g = \text{DTFT}^{-1} \{ \text{DTFT}\{f\} \cdot \text{DTFT}\{g\} \}$$

- Se produce una enorme simplificación si una de las secuencias es N-periódica (por ejemplo “g_N”), porque la $\text{DTFT}\{g_N\}$ es distinta de cero sólo en frecuencias discretas, lo cual deriva en:

$$f * g_N = \text{DTFT}^{-1} \{ \text{DTFT}\{f\} \cdot \text{DTFT}\{g_N\} \} = \text{DFT}^{-1} \{ \text{DFT}\{f_N\} \cdot \text{DFT}\{g_N\} \}$$

Operador integral de Fourier discreto y FFT

Se demuestra (omitimos acá el procedimiento) que la expresión:

$$f * g_N = \text{DTFT}^{-1}\{\text{DTFT}\{f\} \cdot \text{DTFT}\{g_N\}\} = \text{DFT}^{-1}\{\text{DFT}\{f_N\} \cdot \text{DFT}\{g_N\}\}$$

conduce a la “convolución circular” definida como:

$$(f \circledast g)_n = \sum_{m=0}^{N-1} f_m g_{(n-m) \bmod N}.$$

- La convolución circular se computa muy eficientemente con FFT
- Sin embargo, **aparecen efectos de borde indeseables cuando la señal de entrada “f” es no-periódica** que pueden solucionarse aplicando técnicas numéricas como zero-padding, windowing, etc → **Ayuda a tratar problemas con CB no-periódicas (aunque en FNO el tratamiento se hace como sigue)**

Tratamiento de CB no-periódicas

- Recordemos la expresión de la capa de Fourier:

$$v_{t+1}(x) = \sigma(\underbrace{Wv_t(x)}_{\text{mapeo local}} + \underbrace{\mathcal{F}^{-1}[R_\phi(k) \cdot \mathcal{F}\{v_t\}(k)](x)}_{\text{devuelve una salida no-periódica si la señal de entrada es no-periódica (y no se aplicó zero-padding, etc)}})$$

mapeo local que se aplica en el espacio físico (no hereda la restricción de periodicidad)

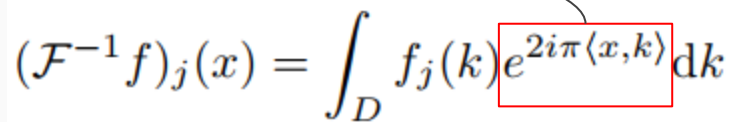
devuelve una salida no-periódica si la señal de entrada es no-periódica (y no se aplicó zero-padding, etc)

W actúa como un término de corrección local y “consciente” de la posición que:

- Modifica los valores en los bordes independientemente del término de Fourier.
- Codifica variaciones espaciales o patrones no periódicos.
- Aprende a cancelar los artefactos de wrap-around del término basado en FFT.

Propiedades generales

- Si bien el operador integral de Fourier pierde los modos de frecuencia alta, **la aplicación de funciones de activación no lineales en el dominio espacial permite recuperar dichos modos** → Las FNO son suficientemente expresivas para modelar operadores fuertemente no lineales
- **Invariancia de resolución** (pueden aprender y evaluar funciones discretizadas de manera arbitraria) → dado que los parámetros son aprendidos directamente en el espacio de Fourier, resolver las funciones en el espacio físico implica proyectar en las bases


$$(\mathcal{F}^{-1}f)_j(x) = \int_D f_j(k) e^{2i\pi\langle x, k \rangle} dk$$

- Esto implica alcanzar “**zero-shot super resolution**”

Propiedades generales

	NN	DeepONets	Interpolation	FNO
Invarianza frente a la discretización	✗	✗	✓	✓
¿La salida es una función?	✗	✓	✓	✓
¿Se puede obtener la salida en cualquier punto?	✗	✓	✓	✓
¿Se puede tomar la entrada en cualquier punto?	✗	✗	✓	✓
Aproximación universal	✗	✓	✗	✓

Algunas desventajas de FNO

- Los datos de entrada (y las salidas) deben corresponder con grillas de puntos estructuradas uniformes
- FNO asume dominios periódicos, por lo que considerar otras configuraciones conduce a artefactos en los bordes
- No es aplicable directamente a geometrías que no sean rectangulares (puede aplicarse zero-padding pero no es eficiente)
- Costo computacional elevado en casos de grillas muy finas o muchos canales, y extensión limitada a problemas 2D/3D
- Manejo limitado de componentes de alta frecuencia, dado el truncamiento aplicado

Ejemplos - Ecuación de Navier-Stokes

$$\begin{aligned}\partial_t w(x, t) + u(x, t) \cdot \nabla w(x, t) &= \nu \Delta w(x, t) + f(x), & x \in (0, 1)^2, t \in (0, T] \\ \nabla \cdot u(x, t) &= 0, & x \in (0, 1)^2, t \in [0, T] \\ w(x, 0) &= w_0(x), & x \in (0, 1)^2\end{aligned}$$

Vorticidad: $w = \nabla \times u$

Coef. de viscosidad: $\nu \in \mathbb{R}_+$

Se pretende aprender el operador que mapea la vorticidad hasta $T=10$ hacia la vorticidad en algún tiempo $T>10$, es decir:

$$w|_{(0,1)^2 \times [0,10]} \mapsto w|_{(0,1)^2 \times (10,T]}$$

Se fijó la resolución de train y test en 64×64

Ejemplos - Ecuación de Navier-Stokes

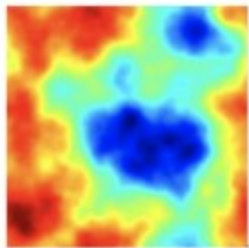
Config	Parameters	Time per epoch	$\nu = 1e-3$	$\nu = 1e-4$	$\nu = 1e-4$	$\nu = 1e-5$
			$T = 50$ $N = 1000$	$T = 30$ $N = 1000$	$T = 30$ $N = 10000$	$T = 20$ $N = 1000$
FNO-3D	6,558,537	38.99s	0.0086	0.1918	0.0820	0.1893
FNO-2D	414,517	127.80s	0.0128	0.1559	0.0834	0.1556
U-Net	24,950,491	48.67s	0.0245	0.2051	0.1190	0.1982
TF-Net	7,451,724	47.21s	0.0225	0.2253	0.1168	0.2268
ResNet	266,641	78.47s	0.0701	0.2871	0.2311	0.2753

FNO-3D: realiza convolución directamente en el dominio espacio-tiempo

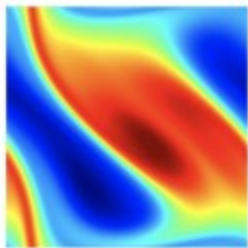
FNO-2D: método FNO 2D con estructura recurrente en el tiempo (FNO+RNN)

Ejemplos - Ecuación de Navier-Stokes

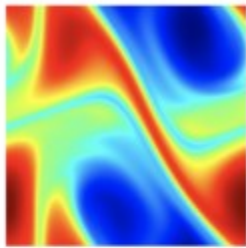
Initial Vorticity



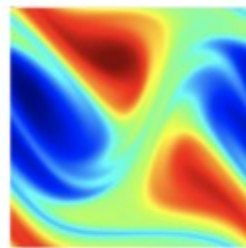
t=15



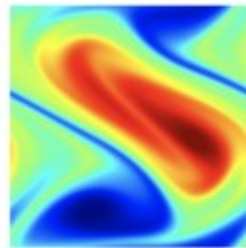
t=20



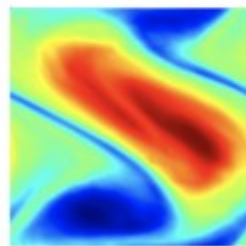
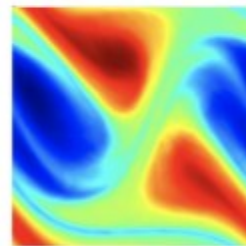
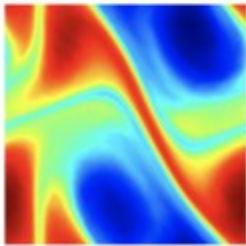
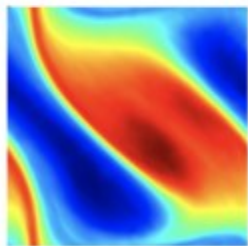
t=25



t=30



Prediction



Zero-shot super-resolution: Navier-Stokes Equation with viscosity $\nu = 1e-4$; Ground truth on top and prediction on bottom; trained on $64 \times 64 \times 20$ dataset; evaluated on $256 \times 256 \times 80$ (see Section 5.4).

Con FNO-3D, el método permite alcanzar zero-shot super-resolution tanto en espacio como en tiempo!!

Aplicabilidad en problemas inversos

$$\mathcal{G} : f \mapsto u \longrightarrow \mathcal{G}^{-1} : u \mapsto f$$

Consideremos la PDE: $\mathcal{L}_\theta[u] = f, \quad u|_{\partial\Omega} = b,$

FNO directo:

$$\min_{\phi} \mathbb{E}_f [\|\mathcal{G}_\phi(f) - u\|^2]$$

FNO inverso:

$$\min_{\psi} \mathbb{E}_u [\|\mathcal{H}_\psi(u) - f\|^2]$$

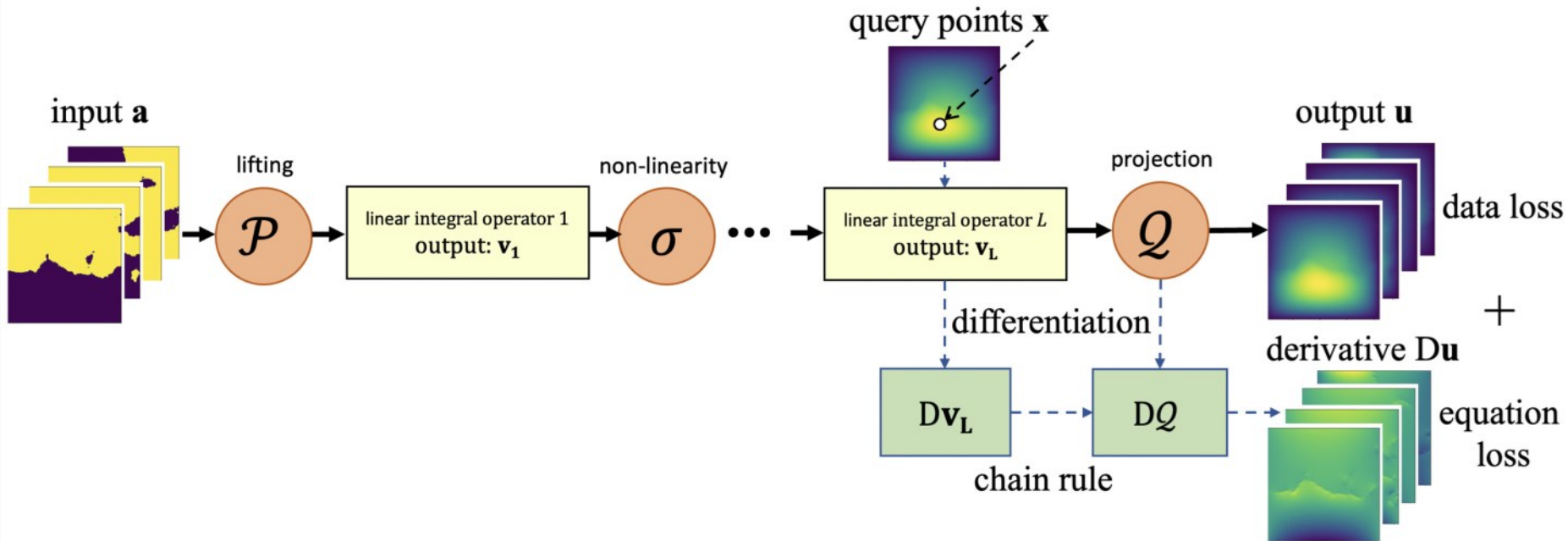
$$\mathcal{H}_\psi \approx \mathcal{G}^{-1}$$

- El operador inverso debe ser bien-colocado (al menos condicionalmente bien-colocado mediante regularización)
- Requerimiento de datos: se requieren pares “(u, f)” para entrenar
- Enfoque híbrido: implica utilizar las PDE como restricciones (para ello necesitamos PINO), tal que:

$$\mathcal{L}_\theta[\mathcal{G}_\phi^{-1}(u)] \approx u$$

PI-FNO (PINO)

- Paper: Physics-Informed Neural Operator for Learning Partial Differential Equations (ACM Journal of Data Science, 2024). Zongyi Li et al.



Estrategia de entrenamiento - 2 etapas

Etapla 1: aprendizaje del operador diferencial

Consiste en aprender el operador $\mathcal{G}_\theta$ para aproximar $\mathcal{G}^\dagger$ utilizando tanto datos rotulados como información de la PDE

Etapla 2: fine-tuning de cada instancia de entrada

Utilizar el operador aprendido previamente $\mathcal{G}_\theta(a)$ para aproximar $u^\dagger$ con la pérdida de la PDE $\mathcal{L}_{pde}$ y un operador de pérdida definido como:

$$\mathcal{L}_{op}(\mathcal{G}_{\theta_i}(a), \mathcal{G}_{\theta_0}(a)) := \|\mathcal{G}_{\theta_i}(a) - \mathcal{G}_{\theta_0}(a)\|_{\mathcal{U}}^2 \longrightarrow \text{Previene que el modelo "olvide" el operador aprendido}$$

"Sí, puedes ajustar G_θ para que se adapte mejor a este problema específico, pero por favor no te alejes demasiado del G_{θ^*} original que funcionaba bien para toda la familia de EDPs."

Obtención de derivadas para las ecuaciones de gobierno

$$D(G_{\theta}a) = \frac{d(G_{\theta}a)}{dx}$$

Computar las derivadas demandadas por la PDE **eficientemente** es un desafío técnico

- **AutoDiff:** no son fácilmente obtenibles las derivadas debido al uso de FFT en FNO (se requiere un tratamiento especial debido a la transformación puntual impuesta por W), y además puede ser prohibitivo el costo computacional
- **Diferencias finitas:** fáciles de implementar, pero sensibles frente a ruido, pérdida de precisión en derivadas de alto orden, y dependen del tamaño de grilla
- **Derivadas espectrales (diferenciación de Fourier):** se computan las derivadas en el espacio de Fourier, de manera exacta. No es apto para problemas fuertemente no lineales y con CB no-periódicas
- **Fourier continuation:** extensión del método de DF para tratar CB no-periódicas

Diferenciación de Fourier

$$\hat{u}_k = \sum_{j=0}^{N-1} u(x_j) e^{-ikx_j}, \quad k = -\frac{N}{2}, \dots, \frac{N}{2} - 1.$$

TF de la secuencia de salida del modelo FNO, en un dominio periódico

$$\frac{d}{dx} (e^{ikx}) = ik e^{ikx}$$

Propiedad de la derivada de la exponencial

$$\widehat{u'}(k) = (ik) \hat{u}(k)$$

La diferenciación en el espacio físico corresponde a una producto por “ik” en el espacio de frecuencias

$$\widehat{u^{(m)}}(k) = (ik)^m \hat{u}(k)$$

Ídem para las derivadas de orden superior

$$u^{(m)}(x) = \mathcal{F}^{-1} \left[(ik)^m \hat{u}(k) \right]$$

Retorno al espacio físico

Conclusiones/Comentarios finales

- FNO como aprendiz de operadores espectrales: aprende mapeos entre espacios de funciones parametrizando operadores integrales en el dominio de Fourier.
- FNO generaliza entre discretizaciones, dominios y resoluciones, lo que lo hace independiente de la malla, ideal para aprender familias de EDP y no sólo instancias particulares (como PINN).
- PINO permite reducir la dependencia de datos etiquetados, mejora la consistencia física y permite entrenar en contextos con escasez de datos.
- **Perspectiva e impacto:** FNO ofrece un marco general para el aprendizaje de operadores, mientras que PINO agrega los priors físicos para garantizar coherencia. Permite acelerar simulaciones al emplearlo como modelo sustituto

¿Dudas o
preguntas?